

# RELAZIONE TECNICA E STRATEGICA DI PROGETTO

**Titolo:** Monitoraggio e Analisi dei Log per Infrastruttura di Backup Centralizzata

**Cliente:** OmniaData Solutions S.p.A.

**Ruolo:** Consulenza Sistemistica Senior GNU/Linux

---

## 1. CONTESTO AZIENDALE E SCENARIO DI CRISI

### Il Cliente: OmniaData Solutions S.p.A.

OmniaData Solutions è una realtà consolidata di medie dimensioni (circa 50 dipendenti) specializzata nello sviluppo di software gestionale per i settori finanziario e assicurativo.

La natura del business impone standard elevatissimi di sicurezza: l'azienda gestisce quotidianamente dati sensibili quali anagrafiche clienti, transazioni bancarie e contratti digitali. Per OmniaData, la continuità operativa e l'integrità dei dati non sono solo requisiti tecnici, ma il fondamento stesso della fiducia accordata dai loro clienti.

### La situazione di crisi (Analisi AS-IS)

L'azienda ha recentemente affrontato un incidente critico che ha messo in luce gravi carenze strutturali. Fino a tre mesi fa, la gestione dei backup era delegata a procedure obsolete ("script amatoriali") ereditate da una gestione precedente e mai aggiornate.

L'infrastruttura è cresciuta in modo disordinato: nuovi server sono stati messi in produzione senza essere integrati nelle procedure di sicurezza.

### Il mandato operativo

A seguito di questo evento, siamo stati ingaggiati con un obiettivo prioritario: "Prendere il controllo del Server di Backup Centralizzato".

Data la particolare topologia di rete, in cui tutti i server (host) sono raggiungibili tramite IP Pubblico statico, il mandato impone un cambio di paradigma rispetto al passato. Non saranno più i singoli server a inviare i dati, ma implementeremo una architettura "Pull" basata su Rsync: sarà il Server Centralizzato a connettersi attivamente agli host remoti per prelevare i dati necessari, garantendo così un controllo centralizzato del traffico e degli accessi.

Il mandato richiede l'implementazione di una soluzione basata esclusivamente su strumenti nativi GNU/Linux (Bash), che rispetti due principi cardine:

1. **Trasparenza:** ogni evento deve essere tracciabile e leggibile attraverso i log.
  2. **Controllabilità:** deve essere sempre possibile ricostruire la storia delle operazioni per fini di conformità.
- 

## 2. ARCHITETTURA TECNICA E DATI

### 2.1 Configurazione del sistema

L'infrastruttura oggetto di analisi è centralizzata su un server basato su sistema operativo GNU/Linux (distribuzione Debian/Ubuntu Server), identificato in rete come `srv-backup-01`. Questa macchina funge da coordinatore unico: riceve, cataloga e archivia i dati provenienti dai nodi della rete. La criticità di questo server impone un monitoraggio dettagliato dell'esito dei trasferimenti, delle performance di rete e della saturazione dello storage.

### 2.2 Infrastruttura IT monitorata

Il sistema gestisce il backup di 8 host, suddivisi in due categorie funzionali:

- **Server Web (`srv-web-XX`):** 5 unità. Ospitano applicazioni front-end. Frequenza critica per garantire il ripristino rapido dei servizi.
- **Server Database (`srv-db-XX`):** 3 unità. Generano dump SQL compressi con alta variabilità dimensionale.

### 2.3 Tecnologia di trasferimento: il ruolo di Rsync

Il "motore" tecnologico scelto per il trasferimento fisico dei dati è **Rsync**.

Questa scelta è standard per l'industria grazie all'algoritmo di "delta-transfer", che minimizza l'uso della banda trasmettendo solo le parti modificate dei file.

Tuttavia, **Rsync è uno strumento di copia, non di gestione**. Sebbene sia eccellente nel muovere i dati, manca nativamente di logica decisionale: non sa gestire policy aziendali e non invia notifiche intelligenti.

È qui che intervengono i nostri script Bash, colmando il divario tra la "forza bruta" di Rsync e le esigenze di business.

## 2.4 Struttura dei log e tecnologia di tracciamento

Per superare i limiti del formato standard (spesso eccessivamente verboso e non strutturato), è stata implementata una serie di logging personalizzata che integra l'affidabilità di **Rsync** con la flessibilità di elaborazione di **Bash**.

### Acquisizione dati e configurazione Rsync

Rsync è stato configurato sfruttando le sue opzioni avanzate di logging (`--out-format`) per esporre esclusivamente le variabili critiche del trasferimento e arricchirle con metadati personalizzati (es. Hostname), garantendo una tracciabilità superiore rispetto ai log nativi.

Le variabili chiave estratte alla fonte sono:

- **%t (Current date time)**: costituisce la base per il timestamp dell'evento.
- **%h (Remote host name)**: fornisce l'indirizzo IP di connessione iniziale.
- **Metadati Custom**: informazioni aggiuntive iniettate durante la configurazione per contestualizzare il log (non disponibili di default).

### Elaborazione e consolidamento

Le metriche generate da Rsync vengono intercettate ed elaborate in tempo reale dallo **Script di Refactoring (Monitor)**. Questo processo trasforma i dati grezzi in un formato strutturato, calcolando lo stato dell'operazione (es. *Successo/Fallimento*) in base ai codici di uscita del processo, anziché affidarsi al parametro `%m` (Module name).

### Formato del record finale

Il risultato è un file di log consolidato, dove ogni record segue uno schema rigoroso:

- **TIMESTAMP**
  - *Descrizione*: data e ora precisa dell'evento (derivato da `%t`).
- **PID (Process ID)**
  - *Descrizione*: identificativo univoco del processo Rsync.
  - *Funzione*: essenziale per associare specifici errori a un determinato host o sessione.
- **HOST / IP PUBBLICO**
  - *Descrizione*: indirizzo risolto dopo la trasformazione.
  - *Logica*: mapping dell'indirizzo locale (es. 127.0.0.1) al corrispondente IP Pubblico fittizio e Hostname associato (es. 201.10.5.88 (srv-web-01)).
- **DIMENSIONE**
  - *Descrizione*: volume dei dati trasferiti nell'operazione (espresso in Byte).
- **PERCORSO\_FILE**
  - *Descrizione*: percorso relativo della risorsa (es. srv-db-02/backup\_1.tar.gz).
  - *Funzione*: dato fondamentale per le logiche di *Cleanup*.
- **ESITO**
  - *Descrizione*: stato finale dell'operazione.
  - *Logica*: valore calcolato dallo script Bash analizzando il return code di Rsync (es. SUCCESS, ERROR\_CONN, ERROR\_PERM, IN\_PROGRESS).

---

### 3. ANALISI DEL CAMBIAMENTO: STATO PRECEDENTE vs SOLUZIONE IMPLEMENTATA

#### La situazione "PRIMA"

La gestione precedente si configurava come "artigianale", presentando gravi vulnerabilità:

- **1. Complessità e illeggibilità dei log**  
Il log nativo di Rsync è un flusso testuale continuo, complesso e non strutturato. Mescola indiscriminatamente messaggi di successo e dati tecnici di debug. Questa disorganizzazione rende impossibile un'analisi rapida e rende fragile qualsiasi tentativo di estrazione/scansione dati automatico, costringendo a "decifrare" faticosamente il formato grezzo per isolare le anomalie.
- **2. Gestione fallimentare delle disconnessioni**  
In assenza di un monitoraggio attivo, se la connessione cade a metà trasferimento, il processo Rsync termina bruscamente ("muore"). Non esiste alcun meccanismo nativo per notificare l'accaduto; il trasferimento risulta semplicemente interrotto e i dati non salvati, lasciando il sistema (e il cliente) all'oscuro del fallimento.
- **3. Saturazione disco da logging**  
Rsync opera in modalità *append* continua, aggiungendo righe al file di log all'infinito senza alcuna logica di archiviazione. Senza intervento esterno, questo comportamento porta inevitabilmente alla saturazione del disco di sistema, compromettendo la stabilità del server.
- **4. Assenza di controllo quote**  
Il motore Rsync copia ciecamente qualsiasi volume di dati gli venga inviato. Non avendo consapevolezza dei limiti contrattuali, permette a un singolo cliente di inviare quantità di dati illimitate (es. 100TB), rischiando di saturare l'intero storage aziendale e causando un disservizio a tutti gli altri utenti.
- **5. Centralizzazione caotica dei dati**  
Rsync genera un unico file di log monolitico in cui si mescolano le operazioni di decine di clienti diversi. Questa commistione dei dati rende estremamente difficile il troubleshooting mirato per un singolo cliente e impedisce la creazione di reportistica specifica.
- **6. Accumulo di dati orfani**  
Quando un trasferimento fallisce o si interrompe, Rsync tende a lasciare sul disco file parziali o corrotti ("orfani"). Questi residui occupano spazio prezioso inutilmente e, accumulandosi nel tempo, riducono l'efficienza dello storage senza apportare valore.
- **7. Mancato backup recente**  
Rsync non è in grado di verificare autonomamente se un cliente possiede un backup COMPLETO valido e recente, rischiando di mantenere solo catene di incrementalі inutilizzabili in fase di restore.
- **8. Vulnerabilità negli accessi**  
Rsync accetta dati da chiunque possieda credenziali tecniche valide, senza verificare lo stato amministrativo del contratto. Ciò significa che non esiste un blocco nativo per

impedire l'accesso a ex-clienti o contratti non più attivi, esponendo il sistema a utilizzi non autorizzati.

- **9. Disallineamento tra servizio e fatturazione**

Rsync continua a erogare il servizio di storage indipendentemente dallo stato dei pagamenti. In assenza di un controllo logico, il sistema continua a consumare risorse per clienti debitori, causando perdite economiche dirette e occupando spazio che potrebbe essere venduto a clienti paganti.

- **10. Rischio di "Falsi Positivi"**

Un codice di uscita SUCCESS da parte di Rsync indica solo che il trasferimento è terminato, ma non garantisce l'integrità fisica del dato sul disco di destinazione. Problemi hardware o fenomeni di "bit rot" possono rendere il file illeggibile nonostante il log riporti un successo, creando una falsa sicurezza sulla ripristinabilità dei dati.

### **Risultato Complessivo:**

Abbiamo trasformato un semplice "trasferimento file" in un **servizio gestito professionale**, riducendo il rischio tecnico e proteggendo il fatturato aziendale attraverso controlli automatici su quote e pagamenti.

### **La situazione “DOPO”**

L'implementazione sviluppata può essere visionata all'interno del repository pubblicato su GitHub.

---

## **4. PROSPETTIVE FUTURE E SCALABILITÀ**

La soluzione progettata risponde non solo alle esigenze attuali, ma è strutturata per sostenere l'espansione futura dell'azienda.

### **Scalabilità tecnica**

L'architettura basata su script Bash garantisce una scalabilità quasi illimitata a costo zero.

- **Indipendenza dai volumi:** la logica di analisi dei log rimane invariata indipendentemente dal numero di righe. L'elaborazione di dati provenienti da 20 server o da 2000 server richiede variazioni di tempo computazionale trascurabili.
- **Assenza di costi di licenza:** a differenza di soluzioni commerciali, l'uso di Rsync combinato con Bash permette di aggiungere infiniti nodi senza costi aggiuntivi.
- **Standardizzazione automatica:** funzionalità come l'ottimizzazione della pianificazione (che sposta automaticamente i backup lenti rilevati da Rsync nel fine settimana) permettono al sistema di auto-bilanciarsi con l'aumento del carico.